

ĐẠI HỌC QUỐC GIA TP.HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN TỔNG HỢP

Hướng Trí tuệ Nhân tạo

Xử lý bài toán Phân loại khối U não
với Đa phương pháp: Naive Bayes, Random Forest,
MultiClassSVM One-vs-One và CNNs

Nhóm 1 - L016 - HK251

Giảng viên hướng dẫn : Trần Nguyễn Minh Duy
Sinh viên: Nguyễn Duy Tân - 2313054
Nguyễn Trần Anh Quân - 2312845
Nguyễn Thành Nam - 2312180
Vũ Huy Gia Khang - 2311486

Thành phố Hồ Chí Minh, 11/2025



Danh sách sinh viên và phân công công việc

STT	Họ và tên	MSSV	Công việc	Đóng góp
1	Nguyễn Duy Tân	2313054	Kiến trúc MultiClassSVM One-vs-One	100%
2	Nguyễn Trần Anh Quân	2312845	Kiến trúc CNN	100%
3	Nguyễn Thành Nam	2312180	Kiến trúc Naive Bayes	100%
4	Vũ Huy Gia Khang	2311486	Kiến trúc Random Forest	100%



Mục lục

1	Giới thiệu đề tài	3
1.1	Lý do chọn đề tài	3
1.2	Mục tiêu đề tài	3
1.3	Phạm vi đề tài	3
2	Các công trình liên quan	3
3	Phương pháp thực hiện	4
3.1	Naive Bayes	4
3.1.1	Cơ sở lý thuyết	4
3.1.2	Gaussian Naive Bayes	5
3.1.3	Multinomial Naive Bayes	5
3.1.4	Bernoulli Naive Bayes	6
3.2	Random Forest	6
3.2.1	Cơ sở lý thuyết	6
3.2.2	Kiến trúc mô hình	7
3.3	MultiClassSVM với phương pháp One-vs-One (OVO)	8
3.3.1	Cơ sở Lý thuyết: Soft-Margin SVM (Nền tảng nhị phân)	8
3.3.2	Kiến trúc Mô hình "One-vs-One" (OVO)	8
3.3.3	Vai trò của Kernel (Kernel Trick)	9
3.4	Convolutional Neural Network	10
3.4.1	Cơ sở lý thuyết	10
3.4.2	Kiến trúc mô hình	11
4	Thực nghiệm trên bộ dữ liệu BRISC	13
4.1	Dataset	13
4.2	Tiền xử lý dữ liệu	13
4.2.1	Tiền xử lý dữ liệu cho các phương pháp Học máy (Naive Bayes, Random Forest và Support Vector Machine)	13
4.2.2	Tiền xử lý dữ liệu cho Convolutional Neural Networks	14
4.3	Huấn luyện các mô hình	14
4.3.1	Huấn luyện mô hình Naive Bayes	14
4.3.2	Huấn luyện mô hình Random Forest	15
4.3.3	Huấn luyện mô hình MultiClassSVM One-vs-One	15
4.3.4	Huấn luyện Convolutinal Neural Networks	16
4.4	Evaluation Metrics	16
4.4.1	Accuracy	17
4.4.2	Precision	17
4.4.3	Recall	17
4.4.4	F1-Score	17
4.5	Kết quả & đánh giá từng mô hình	18
4.5.1	Mô hình Naive Bayes	18
4.5.2	Mô hình Random Forest	19
4.5.3	Mô hình MultiClassSVM One-vs-One	20
4.5.4	Convolutional Neural Networks	20
4.6	Tổng hợp kết quả thực nghiệm	21
5	Kết luận - Phân tích kết quả các mô hình đã sử dụng	22
5.1	Tổng kết kết quả đạt được	22
5.2	Nhận xét và Đánh giá	22
5.3	Hướng phát triển tương lai	23

1 Giới thiệu đề tài

1.1 Lý do chọn đề tài

Trong lĩnh vực y học hiện nay, bệnh u não là một trong những căn bệnh nguy hiểm có tỷ lệ tử vong cao và ảnh hưởng nghiêm trọng đến sức khỏe con người. Việc phát hiện và phân loại chính xác khối u não đóng vai trò then chốt trong quá trình chẩn đoán, điều trị và tiên lượng cho bệnh nhân. Tuy nhiên, các phương pháp chẩn đoán truyền thống chủ yếu dựa vào hình ảnh MRI và đánh giá thủ công của bác sĩ chuyên khoa, do đó dễ chịu ảnh hưởng bởi yếu tố chủ quan và có thể dẫn đến sai sót.

Trước bối cảnh đó, với sự phát triển nhanh chóng của trí tuệ nhân tạo (Artificial Intelligence – AI), học máy (Machine Learning) và học sâu (Deep Learning), đặc biệt là trong lĩnh vực xử lý ảnh y tế, nhiều mô hình và thuật toán tiên tiến đã được ứng dụng nhằm hỗ trợ bác sĩ trong quá trình chẩn đoán. Trong đó, các mô hình tiêu biểu như Support Vector Machine (SVM), Convolutional Neural Network (CNN), Random Forest (RF) là được sử dụng thường xuyên nhất trong lĩnh vực này.

Xuất phát từ thực tiễn đó, nhóm nghiên cứu quyết định lựa chọn đề tài “Phân loại khối u não” nhằm tìm hiểu, so sánh và đánh giá hiệu quả của các phương pháp học máy và học sâu trong việc hỗ trợ phát hiện và chẩn đoán khối u não từ ảnh MRI.

1.2 Mục tiêu đề tài

- Xây dựng và triển khai các mô hình học máy và học sâu (SVM, CNN, Random Forest, Naive Bayes) để thực hiện bài toán phân loại khối u não dựa trên ảnh MRI.
- Đánh giá và so sánh hiệu quả giữa các mô hình thông qua các chỉ số như độ chính xác (Accuracy), độ nhạy (Recall), độ đặc hiệu (Specificity) và chỉ số F1-score.
- Đề xuất mô hình tối ưu nhất cho bài toán phân loại khối u não, nhằm hỗ trợ các chuyên gia y tế trong quá trình phân tích và chẩn đoán hình ảnh.

1.3 Phạm vi đề tài

- Dữ liệu nghiên cứu: Đề tài sử dụng các bộ dữ liệu công khai về ảnh MRI não trên Kaggle: BRISC 2025
- Phạm vi nghiên cứu: Tập trung vào bài toán phân loại hình ảnh MRI theo nhóm có khối u não và không có khối u não. Trong đó, nhóm có khối u não sẽ phân thành 3 loại là glioma, meningioma và pituitary tumor.

2 Các công trình liên quan

Các nghiên cứu gần đây cho thấy mô hình CNN mang lại kết quả tốt trong phân loại ảnh MRI não, giúp nhận diện loại u nhanh và chính xác hơn so với bác sĩ thủ công. Một số nhóm dùng SVM hay Random Forest để so sánh, nhưng các mô hình này phụ thuộc nhiều vào bước trích đặc trưng thủ công nên hiệu quả thấp hơn. Trong thực tế, CNN được xem là lựa chọn phổ biến và hiệu quả nhất cho bài toán phân loại khối u não hiện nay.

3 Phương pháp thực hiện

3.1 Naive Bayes

Naive Bayes (NB) là một thuật toán phân loại xác suất (probabilistic classifier) dựa trên định lý Bayes, được sử dụng rộng rãi trong lĩnh vực học máy (Machine Learning) và khai phá dữ liệu (Data Mining). Mô hình này đặc biệt hiệu quả trong các bài toán phân loại văn bản, lọc thư rác, nhận dạng cảm xúc.

Tên gọi “Naive” (ngây ngô/giả định đơn giản) xuất phát từ giả định độc lập có điều kiện giữa các đặc trưng (features) của dữ liệu — nghĩa là mỗi đặc trưng đóng góp độc lập vào xác suất của lớp kết quả. Nói cách khác, một mô hình Naive Bayes giả định rằng thông tin về lớp do từng biến cung cấp không liên quan đến thông tin từ các biến khác, không có thông tin nào được chia sẻ giữa các biến dự đoán. Tính phi thực tế cao của giả định này, được gọi là giả định độc lập ngây thơ, chính là lý do bộ phân loại này có tên gọi như vậy. Tuy nhiên, chính sự “giả lờ” này giúp mô hình tính toán nhanh, dễ triển khai và đạt hiệu quả tốt trong nhiều trường hợp thực tế.

3.1.1 Cơ sở lý thuyết

Xét bài toán phân loại với C lớp $1, 2, \dots, C$. Giả sử có một điểm dữ liệu $\mathbf{x} \in \mathbb{R}^d$. Hãy tính xác suất để điểm dữ liệu này rơi vào lớp c . Nói cách khác, hãy tính:

$$p(y = c \mid \mathbf{x}) \quad (1)$$

hoặc viết gọn là $p(c \mid \mathbf{x})$.

Tức là tính xác suất để đầu ra là lớp c biết rằng đầu vào là vector $\mathbf{x}$.

Biểu thức này, nếu tính được, sẽ giúp xác định xác suất để điểm dữ liệu rơi vào mỗi lớp. Từ đó có thể xác định lớp của điểm dữ liệu đó bằng cách chọn lớp có xác suất cao nhất:

$$c^* = \arg \max_{c \in \{1, \dots, C\}} p(c \mid \mathbf{x}) \quad (2)$$

Biểu thức (2) thường khó tính trực tiếp. Thay vào đó, quy tắc Bayes được sử dụng:

$$c^* = \arg \max_c p(c \mid \mathbf{x}) \quad (3)$$

$$= \arg \max_c \frac{p(\mathbf{x} \mid c) p(c)}{p(\mathbf{x})} \quad (4)$$

$$= \arg \max_c p(\mathbf{x} \mid c) p(c) \quad (5)$$

Trong đó (3) $\rightarrow$ (4) là do quy tắc Bayes, và (4) $\rightarrow$ (5) vì mẫu số $p(\mathbf{x})$ không phụ thuộc vào c .

Tiếp tục xét biểu thức (5), $p(c)$ có thể hiểu là xác suất để một điểm rơi vào lớp c . Giá trị này thường được tính bằng Maximum Likelihood Estimation (MLE), tức tỉ lệ số điểm trong tập huấn luyện thuộc lớp c chia cho tổng số điểm; hoặc cũng có thể dùng Maximum A Posteriori (MAP).

Thành phần còn lại $p(\mathbf{x} \mid c)$ — tức phân phối của các điểm dữ liệu trong lớp c — thường rất khó tính vì $\mathbf{x}$ là biến ngẫu nhiên nhiều chiều, yêu cầu rất nhiều dữ liệu huấn luyện để ước lượng. Để đơn giản, người ta giả sử rằng các thành phần của $\mathbf{x}$ là độc lập nếu biết lớp c , tức:

$$p(\mathbf{x} \mid c) = p(x_1, x_2, \dots, x_d \mid c) = \prod_{i=1}^d p(x_i \mid c) \quad (6)$$

Giả thiết rằng các chiều dữ liệu độc lập với nhau nếu biết lớp là quá chặt và ít khi được thoả mãn thật sự. Tuy nhiên giả thiết “ngây ngô” này lại mang lại kết quả tốt trong thực tế. Giả thiết này

được gọi là *Naive Bayes*. Cách xác định lớp của dữ liệu dựa trên giả thiết này được gọi là *Naive Bayes Classifier (NBC)*. NBC nhờ vào tính đơn giản mà có tốc độ huấn luyện và kiểm thử rất nhanh, giúp nó hiệu quả trong các bài toán *large-scale*.

Trong bước huấn luyện, các phân phối $p(c)$ và $p(x_i | c)$, $i = 1, \dots, d$ sẽ được ước lượng từ dữ liệu huấn luyện, thường bằng MLE hoặc MAP. Ở bước kiểm thử, với một điểm mới $\mathbf{x}$, lớp của nó sẽ được xác định bởi:

$$c^* = \arg \max_{c \in \{1, \dots, C\}} p(c) \prod_{i=1}^d p(x_i | c) \quad (7)$$

Khi d lớn và các xác suất nhỏ, vế phải của (7) sẽ là một số rất nhỏ, dễ gặp sai số về mặt tính toán. Vì vậy biểu thức (7) thường được viết lại dưới dạng log của vế phải:

$$c^* = \arg \max_{c \in \{1, \dots, C\}} \left[\log p(c) + \sum_{i=1}^d \log p(x_i | c) \right] \quad (7.1)$$

Việc này không ảnh hưởng đến kết quả vì hàm log là hàm đồng biến trên tập các số dương.

3.1.2 Gaussian Naive Bayes

Mô hình này dùng cho dữ liệu liên tục. Với mỗi chiều dữ liệu i và lớp c , giả sử x_i tuân theo phân phối chuẩn với kỳ vọng μ_{ci} và phương sai σ_{ci}^2 :

$$p(x_i | c) = p(x_i | \mu_{ci}, \sigma_{ci}^2) = \frac{1}{\sqrt{2\pi} \sigma_{ci}} \exp\left(-\frac{(x_i - \mu_{ci})^2}{2\sigma_{ci}^2}\right) \quad (8)$$

Bộ tham số $\theta = \{\mu_{ci}, \sigma_{ci}^2\}$ được xác định bằng Maximum Likelihood:

$$(\mu_{ci}, \sigma_{ci}^2) = \arg \max_{\mu_{ci}, \sigma_{ci}^2} \prod_{n=1}^N p(x_i^{(n)} | \mu_{ci}, \sigma_{ci}^2) \quad (9)$$

3.1.3 Multinomial Naive Bayes

Mô hình này thường dùng trong phân loại văn bản, nơi mỗi văn bản được biểu diễn theo mô hình Bag of Words với vector độ dài d (số từ trong từ điển), và x_i là số lần từ thứ i xuất hiện. Khi đó:

$$\lambda_{ci} = p(x_i | c) = \frac{N_{ci}}{N_c} \quad (10)$$

trong đó:

- N_{ci} là tổng số lần từ thứ i xuất hiện trong các văn bản thuộc lớp c .
- N_c là tổng số từ (bao gồm lặp) trong các văn bản thuộc lớp c .

Tổng $\sum_{i=1}^d \lambda_{ci} = 1$. Để tránh trường hợp xác suất bằng 0 khi một từ chưa từng xuất hiện, ta dùng kỹ thuật *Laplace smoothing*:

$$\hat{\lambda}_{ci} = \frac{N_{ci} + \alpha}{N_c + d\alpha} \quad (11)$$

với $\alpha > 0$, thường chọn $\alpha = 1$. Khi đó mỗi lớp c được mô tả bởi bộ xác suất

$$\hat{\lambda}_c = \{\hat{\lambda}_{c1}, \dots, \hat{\lambda}_{cd}\}.$$

3.1.4 Bernoulli Naive Bayes

Mô hình này áp dụng cho dữ liệu nhị phân (0 hoặc 1). Ví dụ: trong bài toán văn bản, thay vì đếm số lần xuất hiện của từ, chỉ quan tâm xem từ đó có xuất hiện hay không trong văn bản. Khi đó:

$$p(x_i | c) = p(i | c)^{x_i} (1 - p(i | c))^{1-x_i}$$

với $p(i | c)$ là xác suất từ thứ i xuất hiện trong các văn bản của lớp c .

3.2 Random Forest

Random Forest là một thuật toán học máy thuộc nhóm *ensemble learning*, được đề xuất bởi Leo Breiman (2001) [1]. Phương pháp này dựa trên nguyên lý kết hợp nhiều cây quyết định (*decision trees*) độc lập để tạo nên một mô hình mạnh và ổn định hơn.

Trong quá trình huấn luyện, thuật toán tạo ra một tập hợp các cây quyết định, mỗi cây được huấn luyện trên một tập con dữ liệu được chọn ngẫu nhiên có hoàn lại từ tập huấn luyện gốc (*bootstrap sampling*). Tại mỗi nút chia trong cây, chỉ một tập con ngẫu nhiên các đặc trưng được xét đến thay vì toàn bộ, nhằm giảm tương quan giữa các cây và tăng tính đa dạng cho rừng [8]. Khi dự đoán, mỗi cây trong rừng bỏ một phiếu cho nhãn lớp, và lớp có tổng số phiếu cao nhất được chọn là kết quả cuối cùng.

Cách tiếp cận “nhiều cây nhỏ tạo nên một rừng lớn” giúp Random Forest giảm phương sai (variance), hạn chế overfitting, và tăng độ chính xác tổng thể của hệ thống. Phương pháp này đã được áp dụng rộng rãi trong nhiều lĩnh vực. Nhờ tính ổn định, khả năng mở rộng và độ chính xác cao, Random Forest được xem là một trong những mô hình tiêu chuẩn cho các bài toán phân loại phi tuyến có kích thước dữ liệu trung bình đến lớn.

3.2.1 Cơ sở lý thuyết

Một mô hình Random Forest gồm N cây quyết định độc lập:

$$RF = \{h(x, \theta_k), k = 1, 2, \dots, N\}$$

trong đó:

- x là vector đặc trưng đầu vào;
- θ_k là vector ngẫu nhiên sinh ra ở mỗi cây (bao gồm các mẫu dữ liệu và tập đặc trưng được chọn ngẫu nhiên);
- $h(x, \theta_k)$ là cây phân loại được huấn luyện bằng dữ liệu sinh bởi θ_k .

Khi dự đoán đối với bài toán phân loại, kết quả đầu ra là:

$$\hat{y} = \text{mode}\{h(x, \theta_k)\}, \quad k = 1, \dots, N$$

Quá trình huấn luyện của Random Forest bao gồm hai bước chính:

1. *Bootstrap Sampling* (lấy mẫu có hoàn lại).

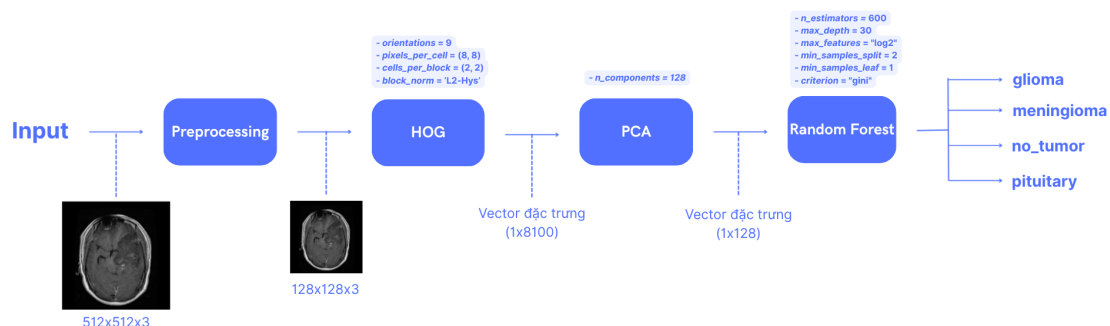
- **Mục tiêu:** Tạo ra các tập huấn luyện con độc lập cho từng cây, giúp các cây nhìn thấy các biến thể khác nhau của dữ liệu, qua đó giảm tương quan giữa các cây.
- **Cách thực hiện:** Với tập huấn luyện gốc $D = \{(x_i, y_i)\}_{i=1}^n$, để huấn luyện cây thứ k , ta rút ngẫu nhiên có hoàn lại n lần từ D để thu được tập con D_k (kích thước n , nhưng các phần tử có thể lặp).
- **Lợi ích:** Tăng tính đa dạng giữa các cây, giảm phương sai tổng thể của mô hình mà không làm tăng nhiều độ phức tạp.

2. Random Feature Selection (chọn tập con đặc trưng tại mỗi nút).

- **Mục tiêu:** Giảm tương quan giữa các cây bằng cách buộc mỗi lần tách nút chỉ được cân nhắc trên một tập con nhỏ đặc trưng, tránh việc các cây lặp lại cùng một đặc trưng mạnh.
- **Cách thực hiện tại mỗi nút:**
 - Tại mỗi nút trong cây, chọn ngẫu nhiên m đặc trưng trong tổng số M đặc trưng của dữ liệu.
 - Xác định đặc trưng và ngưỡng chia tốt nhất (ví dụ: *Gini*, *Entropy* cho phân loại; *MSE* cho hồi quy) trong m đặc trưng được chọn.
 - Chia dữ liệu thành các nhánh con dựa trên đặc trưng đó.
- **Cách chọn số lượng đặc trưng m (**max_features**):**
 - Với bài toán phân loại, thường chọn $m = \sqrt{M}$.
 - Với bài toán dự đoán giá trị (hồi quy), thường chọn $m = \frac{M}{3}$.
 - Nếu m quá nhỏ, các cây sẽ khác nhau nhiều nhưng học được ít thông tin hơn; nếu m quá lớn, các cây sẽ giống nhau hơn. Vì vậy cần chọn giá trị m hợp lý.
- **Lợi ích:** Việc chọn đặc trưng ngẫu nhiên khiến mỗi cây đưa ra quyết định theo cách khác nhau. Khi kết hợp nhiều cây như vậy, mô hình cuối cùng trở nên chính xác và ít bị ảnh hưởng bởi dữ liệu nhiễu.

Thuật toán được gọi là Random Forest nhờ hai yếu tố ngẫu nhiên là Bootstrap Sampling và Random Feature Selection. Sự kết hợp này giúp mô hình trở nên ổn định hơn, giảm sai số và hạn chế hiện tượng overfitting, đồng thời cho kết quả chính xác hơn nhiều so với khi chỉ sử dụng một cây quyết định đơn lẻ.

3.2.2 Kiến trúc mô hình



Hình 1: Kiến trúc mô hình Random Forest

3.3 MultiClassSVM với phương pháp One-vs-One (OVO)

Support Vector Machines (SVM)[2, 15] về bản chất là một thuật toán được thiết kế cho bài toán phân lớp nhị phân (binary classification). Mục tiêu của SVM là tìm ra một siêu phẳng (hyperplane) tối ưu, không chỉ phân chia hai lớp dữ liệu mà còn tối đa hóa khoảng cách (lề - margin) giữa các điểm gần nhất của hai lớp (gọi là các support vector). Để áp dụng SVM cho bài toán phân lớp đa lớp (multi-class) với k lớp ($k > 2$), các phương pháp mở rộng là cần thiết. Có hai hướng tiếp cận chính:

- **Kết hợp các bộ phân lớp nhị phân:** Huấn luyện nhiều bộ phân lớp SVM nhị phân và tổng hợp kết quả của chúng. Các chiến lược phổ biến bao gồm "One-vs-One" (OVO) và "One-vs-Rest" (OVR).
- **Phương pháp "All-Together":** Xây dựng một bài toán tối ưu duy nhất để xem xét tất cả k lớp cùng một lúc, giải quyết vấn đề một cách trực tiếp.

Trong đó, chiến lược "One-vs-One" (OVO)[9] là một phương pháp hiệu quả, thực tiễn và được triển khai rộng rãi, tiêu biểu là trong thư viện `sklearn.svm.SVC`.

3.3.1 Cơ sở Lý thuyết: Soft-Margin SVM (Nền tảng nhị phân)

Kiến trúc OVO dựa trên việc xây dựng nhiều bộ phân lớp SVM nhị phân. Dữ liệu trong thực tế (như các đặc trưng HOG của ảnh) thường không thể phân tách tuyến tính một cách hoàn hảo (non-separable). Do đó, mỗi bộ phân lớp này là một mô hình Soft-Margin SVM[15], dựa trên công thức "lề mềm" của Cortes và Vapnik (1995). Bài toán tối ưu (primal problem) cho mỗi bộ phân lớp nhị phân là:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^l \xi_i$$

Tuân theo các ràng buộc:

$$\begin{aligned} y_i(w^T \phi(x_i) + b) &\geq 1 - \xi_i, \quad \forall i = 1, \dots, l \\ \xi_i &\geq 0, \quad \forall i = 1, \dots, l \end{aligned}$$

Trong đó:

- x_i là vector đặc trưng của mẫu thứ i (đã được ánh xạ qua ϕ).
- $y_i \in \{-1, 1\}$ là nhãn của lớp nhị phân.
- w và b định nghĩa siêu phẳng phân lớp.
- ξ_i (biến bù - slack variables) là các biến cho phép một số điểm dữ liệu vi phạm lề (margin) hoặc thậm chí bị phân loại sai.
- C là tham số phạt (penalty parameter). Đây là siêu tham số (hyperparameter) cân bằng giữa hai mục tiêu: (1) Tối đa hóa lề (tối thiểu hóa $\|w\|^2$) và (2) Tối thiểu hóa lỗi huấn luyện (tối thiểu hóa $\sum \xi_i$).

3.3.2 Kiến trúc Mô hình "One-vs-One" (OVO)

Chiến lược OVO [9] xây dựng một bộ phân lớp nhị phân cho mỗi cặp lớp phân biệt [7].

Giai đoạn Huấn luyện (Training)

Với k lớp, mô hình sẽ xây dựng tổng cộng $k(k-1)/2$ bộ phân lớp SVM. Trong trường hợp $k = 4$ (glioma, meningioma, no_tumor, pituitary), mô hình sẽ huấn luyện 6 bộ phân lớp nhị phân riêng biệt. Cụ thể như sau:

1. SVM (glioma vs. meningioma)
2. SVM (glioma vs. no_tumor)
3. SVM (glioma vs. pituitary)
4. SVM (meningioma vs. no_tumor)
5. SVM (meningioma vs. pituitary)
6. SVM (no_tumor vs. pituitary)

Mỗi bộ phân lớp SVM_{ij} (ví dụ: glioma vs. meningioma) chỉ được huấn luyện trên tập con của dữ liệu, tức là chỉ các mẫu x_t có nhãn $y_t = i$ hoặc $y_t = j$. Mỗi bộ phân lớp này sẽ giải bài toán tối ưu Soft-Margin SVM để tìm ra siêu phẳng w^{ij} và b^{ij} tối ưu của riêng nó.

Giai đoạn Dự đoán (Prediction) - Chiến lược Bỏ phiếu "Max Wins"

Kiến trúc dự đoán của OVO là một hệ thống bỏ phiếu (voting strategy), xem Thuật toán 1.

Algorithm 1 OVO_PREDICT

Require: Tập các bộ phân lớp nhị phân đã huấn luyện $\{SVM_{ij}\}$ cho mọi cặp $1 \leq i < j \leq k$;
vector đặc trưng x_{test}

Ensure: Nhãn dự đoán $\hat{y}$

- 1: Khởi tạo mảng phiếu $v[1 \dots k] \leftarrow 0$
- 2: Khởi tạo mảng điểm tổng hợp $S_{\text{agg}}[1 \dots k] \leftarrow 0$
- 3: **for** $1 \leq i < j \leq k$ **do**
- 4: Tính giá trị hàm quyết định: $s_{ij} \leftarrow (w^{ij})^\top \phi(x_{\text{test}}) + b^{ij}$
- 5: **if** $s_{ij} \geq 0$ **then**
- 6: $v[i] \leftarrow v[i] + 1$ {bỏ phiếu cho lớp i }
- 7: $S_{\text{agg}}[i] \leftarrow S_{\text{agg}}[i] + |s_{ij}|$ {cộng điểm tin cậy}
- 8: **else**
- 9: $v[j] \leftarrow v[j] + 1$ {bỏ phiếu cho lớp j }
- 10: $S_{\text{agg}}[j] \leftarrow S_{\text{agg}}[j] + |s_{ij}|$ {cộng điểm tin cậy}
- 11: **end if**
- 12: **end for**
- 13: $m \leftarrow \max_{r \in \{1, \dots, k\}} v[r]$ {số phiếu cao nhất}
- 14: $\mathcal{M} \leftarrow \{r \mid v[r] = m\}$ {tập nhãn đạt tối đa phiếu}
- 15: **if** $|\mathcal{M}| = 1$ **then**
- 16: $\hat{y} \leftarrow$ the unique element of $\mathcal{M}$
- 17: **else**
- 18: **Tie-breaker:** Chọn nhãn có tổng điểm tin cậy lớn nhất:

$$\hat{y} \leftarrow \arg \max_{r \in \mathcal{M}} S_{\text{agg}}[r]$$

{nếu vẫn hoà, chọn nhãn có chỉ số nhỏ nhất}

19: **end if**

20: **return** $\hat{y}$

3.3.3 Vai trò của Kernel (Kernel Trick)

Bài toán tối ưu SVM (cả primal và dual) phụ thuộc vào phép toán tích vô hướng (dot-product) $\phi(x_i)^T \phi(x_j)$ giữa các vector đặc trưng trong không gian đặc trưng (feature space).

Kernel Trick [2, 17, 13] là một kỹ thuật thay thế phép toán tích vô hướng này bằng một hàm kernel $K(x_i, x_j)$. Việc này tương đương với việc âm thầm ánh xạ (map) dữ liệu x từ không gian

đặc trưng ban đầu sang một không gian đặc trưng mới $\phi(x)$ có số chiều rất cao (thậm chí vô hạn).

Trong không gian mới này, dữ liệu có khả năng trở nên phân tách tuyến tính cao hơn, cho phép siêu phẳng w phân chia các lớp hiệu quả hơn.

Một trong những kernel mạnh mẽ và phổ biến nhất là **Radial Basis Function (RBF) Kernel**[13]:

$$K(x, z) = \exp(-\gamma \|x - z\|^2)$$

Tham số $\gamma > 0$ kiểm soát độ "linh hoạt" của đường biên quyết định. Nếu γ nhỏ, đường biên sẽ mượt hơn (tương tự tuyến tính); nếu γ lớn, đường biên sẽ phức tạp và bám sát vào dữ liệu huấn luyện hơn.

Kiến trúc mô hình OVO là một tập hợp $k(k-1)/2$ bộ phân lớp Soft-Margin SVM, trong đó mỗi bộ sử dụng một hàm Kernel (như RBF) để tìm ra một siêu phẳng tối ưu. Kết quả dự đoán cuối cùng được quyết định bằng chiến lược bỏ phiếu "Max Wins".

3.4 Convolutional Neural Network

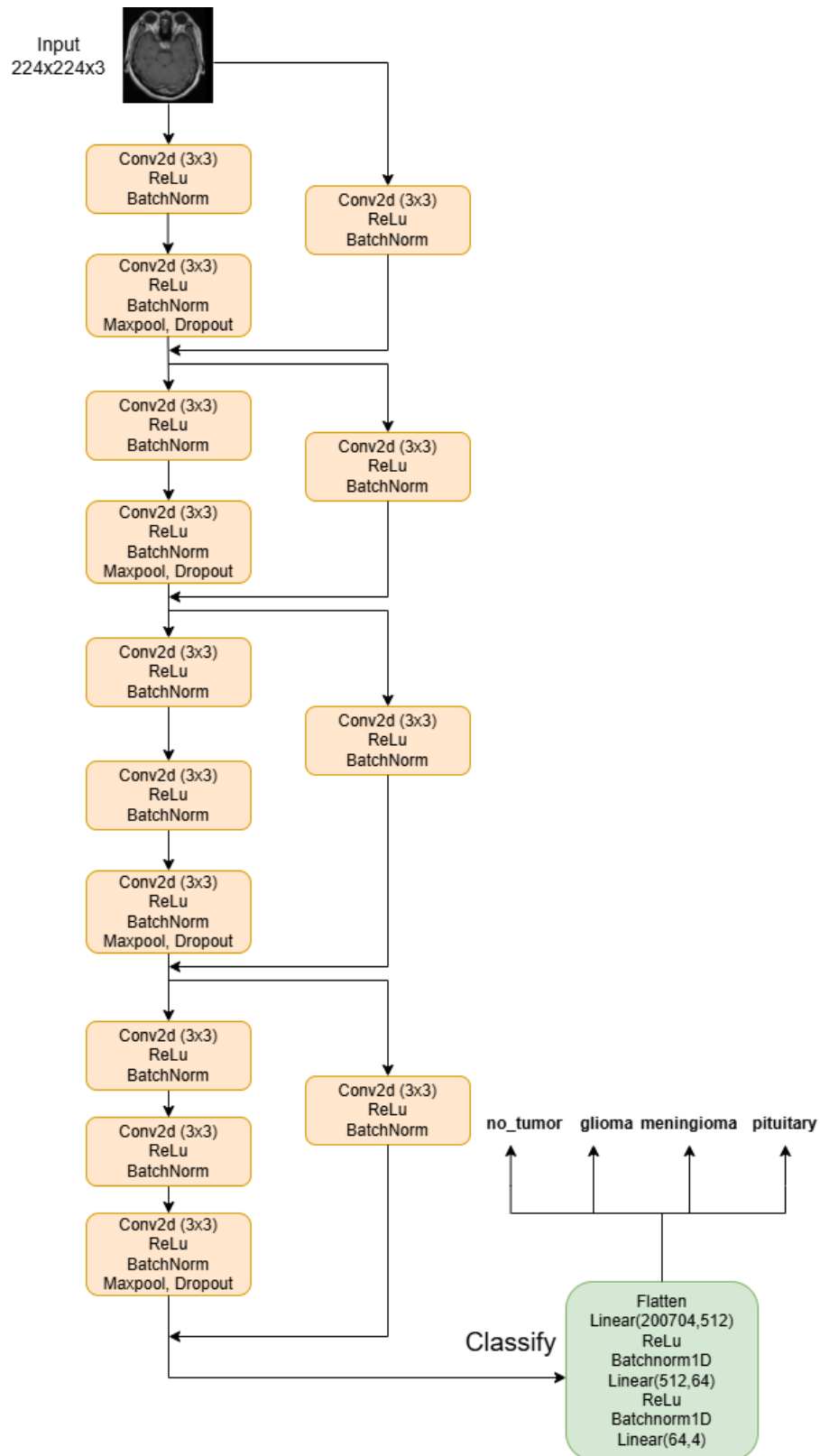
3.4.1 Cơ sở lý thuyết

- **Mạng nơ-ron tích chập (Convolutional Neural Network - CNN)** là một kiến trúc mạng học sâu được thiết kế đặc biệt cho xử lý dữ liệu dạng lưới, như ảnh hoặc video. CNN hoạt động dựa trên các phép **tích chập (convolution)** để tự động trích xuất đặc trưng từ dữ liệu đầu vào mà không cần thiết kế thủ công.
- Kiến trúc cơ bản của một CNN thường bao gồm các thành phần:
 - **Lớp Convolution (Conv Layer):** thực hiện phép tích chập giữa bộ lọc (kernel) và ảnh đầu vào để phát hiện đặc trưng cục bộ.
 - **Lớp Pooling:** giảm kích thước không gian của đặc trưng, giúp giảm số lượng tham số và hạn chế overfitting.
 - **Lớp Activation (ReLU):** giới thiệu tính phi tuyến vào mô hình, giúp mạng học các mối quan hệ phức tạp.
 - **Lớp Fully Connected (FC):** kết hợp các đặc trưng đã học để thực hiện phân loại cuối cùng.
- CNN đặc biệt hiệu quả trong các bài toán **phân loại ảnh (image classification)**, **nhận dạng vật thể (object recognition)** và **chẩn đoán y học dựa trên ảnh**. Trong bài toán phân loại khối u não, CNN có khả năng tự động nhận diện các đặc trưng hình dạng và cấu trúc bất thường trong ảnh MRI.
- Một số mô hình CNN kinh điển được sử dụng rộng rãi:
 - **LeNet-5** [16] – mô hình CNN đầu tiên cho nhận dạng chữ viết tay.
 - **AlexNet** [4] – đánh dấu bước ngoặt của Deep Learning trong thị giác máy tính.
 - **VGGNet** [11] – sử dụng nhiều lớp tích chập nhỏ (3×3) để tăng độ sâu mô hình.
 - **EfficientNet** [10] – tối ưu đồng thời chiều sâu, chiều rộng và độ phân giải của mô hình.
- Nhờ khả năng trích xuất đặc trưng tự động, CNN trở thành nền tảng của hầu hết các hệ thống thị giác máy tính hiện nay và được ứng dụng rộng rãi trong lĩnh vực y học, đặc biệt là phân tích ảnh MRI để phát hiện và phân loại khối u não.



3.4.2 Kiến trúc mô hình

Lấy cảm hứng từ kỹ thuật *skip connection* [12] của mô hình ResNet [14], nhóm đã phát triển một mô hình CNN tự thiết kế có kiến trúc như Hình 2.



Hình 2: Kiến trúc mô hình CNN tự thiết kế

4 Thực nghiệm trên bộ dữ liệu BRISC

4.1 Dataset

- Bộ dữ liệu sử dụng là **BRISC 2025 Dataset**, được lấy từ Kaggle tại địa chỉ: [Link dataset](#).
- Nhóm chỉ sử dụng phần **task Classification** của bộ dữ liệu.
- Dữ liệu bao gồm ảnh **MRI não người**, được chia thành bốn lớp: *no_tumor*, *glioma*, *meningioma*, và *pituitary*.
- Ảnh được lưu trong các thư mục riêng tương ứng với từng lớp, thuận tiện cho việc đọc và huấn luyện mô hình.
- Tổng số ảnh : **6000 tấm**, định dạng chủ yếu là .jpg.
- Kích thước ảnh gốc **512×512 pixel**, chỉ gồm ảnh màu.
- Ảnh được chụp từ nhiều góc độ và lát cắt MRI khác nhau, thể hiện rõ cấu trúc vùng não và khối u.
- Một số ảnh có **nhiều hoặc độ sáng không đồng đều**, phản ánh dữ liệu y tế thực tế.
- Số lượng ảnh giữa các lớp **không hoàn toàn cân bằng**.
- Bộ dữ liệu có **cấu trúc rõ ràng, đa dạng và phù hợp** cho bài toán phân loại khối u não.

4.2 Tiền xử lý dữ liệu

4.2.1 Tiền xử lý dữ liệu cho các phương pháp Học máy (Naive Bayes, Random Forest và Support Vector Machine)

Histogram of Oriented Gradients (HOG)

Histogram of Oriented Gradients (HOG) [3] là đặc trưng mô tả hình dạng dựa trên phân bố hướng gradient cục bộ, đặc biệt hiệu quả cho bài toán nhận dạng người. Với ảnh đầu vào $I(x, y)$, gradient được tính bằng đạo hàm bậc nhất:

$$g_x = I(x+1, y) - I(x-1, y), \quad g_y = I(x, y+1) - I(x, y-1),$$

giá trị độ lớn và hướng:

$$m = \sqrt{g_x^2 + g_y^2}, \quad \theta = \arctan \frac{g_y}{g_x}.$$

Ảnh được chia thành các *cells*, mỗi cell tạo histogram hướng từ các giá trị m theo θ . Các cell được gom thành *blocks* và chuẩn hoá để tăng bất biến sáng và độ tương phản, ví dụ chuẩn hoá L_2 :

$$v' = \frac{v}{\sqrt{\|v\|_2^2 + \epsilon^2}}.$$

Các block được lấy chồng lấn, toàn bộ vector HOG được ghép lại và đưa vào bộ phân loại tuyến tính (thường là SVM). HOG mạnh trong việc mô tả cấu trúc biên và hướng cạnh, ổn định trước thay đổi ánh sáng và biến dạng nhỏ của vật thể.

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) [6] là phương pháp giảm chiều dữ liệu bằng cách tìm các tổ hợp tuyến tính của biến gốc sao cho giữ lại tối đa phương sai. Cho ma trận dữ liệu $X \in \mathbb{R}^{n \times d}$ đã được chuẩn hoá (các cột có trung bình 0), PCA tìm các vectơ riêng của ma trận hiệp phương

sai $\Sigma = \frac{1}{n}X^T X$. Các trục chính (principal components) được lấy theo thứ tự giảm dần của giá trị riêng, tạo nên ánh xạ thấp chiều:

$$Z = XV,$$

với $V = [v_1, \dots, v_k]$ là ma trận gồm k vectơ riêng ứng với k trị riêng lớn nhất. Mỗi trị riêng λ_i biểu diễn lượng phương sai giải thích trên trục i . PCA tương đương với phân rã trị riêng (EVD) hoặc SVD của X :

$$X = U\Sigma V^T, \quad \text{PCs} = U\Sigma.$$

PCA bảo toàn cấu trúc dữ liệu trong không gian thấp chiều, giảm nhiễu và là bước tiền xử lý phổ biến trong nhận dạng mẫu và học máy.

4.2.2 Tiền xử lý dữ liệu cho Convolutional Neural Networks

- Dữ liệu huấn luyện và kiểm thử được xử lý bằng thư viện **Torchvision Transforms**.
- Đối với **tập huấn luyện (train set)**, nhóm áp dụng nhiều kỹ thuật **data augmentation** nhằm tăng tính đa dạng của dữ liệu và giảm hiện tượng overfitting:
 - `RandomCrop(32, padding=2)`: cắt ngẫu nhiên vùng 32×32 pixel trong ảnh, giúp mô hình học được đặc trưng ở nhiều vị trí khác nhau.
 - `RandomHorizontalFlip(p=0.5)`: lật ngang ảnh với xác suất 50%, tăng khả năng khái quát hóa của mô hình.
 - `RandomRotation(5)`: xoay ảnh trong khoảng $\pm 5^\circ$, giúp mô hình ổn định với góc chụp khác nhau.
 - `Resize(224, 224)`: chuẩn hóa kích thước ảnh đầu vào cho phù hợp với mô hình CNN.
 - `ToTensor()`: chuyển ảnh sang dạng tensor để sử dụng trong PyTorch.
 - `Normalize(mean, std)`: chuẩn hóa giá trị pixel theo chuẩn của ImageNet, giúp quá trình huấn luyện nhanh và ổn định hơn.
 - `RandomErasing(p=0.75)`: che ngẫu nhiên một vùng nhỏ trong ảnh với xác suất 75%, giúp mô hình giảm phụ thuộc vào các vùng cố định.
- Đối với **tập kiểm thử (test set)**, chỉ áp dụng các bước cơ bản:
 - `Resize(224, 224)`, `ToTensor()`, và `Normalize()` để đảm bảo dữ liệu đầu vào có cùng định dạng và tỷ lệ như tập huấn luyện.
- Sau khi tiền xử lý, dữ liệu được nạp bằng `DataLoader` với kích thước batch là 32, `shuffle=True` cho tập huấn luyện và `shuffle=False` cho tập kiểm thử.
- Quá trình tiền xử lý giúp đảm bảo dữ liệu đầu vào được chuẩn hóa, đa dạng và phù hợp cho việc huấn luyện mô hình học sâu **CNN**.

4.3 Huấn luyện các mô hình

4.3.1 Huấn luyện mô hình Naive Bayes

Các thông số huấn luyện

- Mô hình được huấn luyện sử dụng thuật toán *Gaussian Naive Bayes (GaussianNB)* – một biến thể của Naive Bayes giả định rằng các đặc trưng tuân theo phân phối chuẩn (Gaussian).
- Do đặc trưng của Naive Bayes, mô hình không yêu cầu các siêu tham số như *learning rate*, *batch size*, *optimizer* hay *số epoch*, vì toàn bộ tập dữ liệu được xử lý một lần thông qua công thức xác suất.

- Quá trình huấn luyện bao gồm:
 - Giảm chiều dữ liệu bằng PCA từ không gian đặc trưng HOG ban đầu xuống 100 chiều.
 - Huấn luyện mô hình GaussianNB trên tập đặc trưng đã giảm chiều ($X_{\text{train_pca}}$).
 - Dự đoán kết quả trên tập kiểm thử ($X_{\text{test_pca}}$).
- Sau khi huấn luyện, mô hình được đánh giá bằng các chỉ số:
 - Độ chính xác (*Accuracy*)
 - Báo cáo phân loại (*Precision, Recall, F1-score*)
 - Ma trận nhầm lẫn (*Confusion Matrix*)
- Kích thước đặc trưng đầu vào sau PCA là 100, giúp mô hình học nhanh, tránh *overfitting* và giảm chi phí tính toán.
- Việc kết hợp ba bước **HOG** $\rightarrow$ **PCA** $\rightarrow$ **GaussianNB** giúp hệ thống đạt được cân bằng giữa độ chính xác và tốc độ xử lý, phù hợp cho các bài toán nhận dạng ảnh cơ bản.

4.3.2 Huấn luyện mô hình Random Forest

Do đặc trưng của Random Forest, mô hình không cần các siêu tham số mạng nơ-ron như *learning rate, batch size, optimizer* hay *epochs*. Các siêu tham số quan trọng gồm:

- `n_estimators`: số lượng cây quyết định trong rừng.
- `max_depth`: độ sâu tối đa của mỗi cây.
- `max_features`: số đặc trưng được chọn ngẫu nhiên khi chia một nút.
- `min_samples_split`: số lượng mẫu tối thiểu cần thiết để chia một nút.
- `min_samples_leaf`: số lượng mẫu tối thiểu trong mỗi nút lá (node cuối cùng).
- `criterion = "gini"`: chỉ số dùng để đo độ thuần khiết (impurity) của các mẫu tại một nút: $G(t) = 1 - \sum_{i=1}^C [p(i|t)]^2$, với C là tổng số lớp và $p(i|t)$ là tỉ lệ mẫu thuộc lớp i tại nút t .

Quy trình huấn luyện bao gồm:

- Trích đặc trưng HOG từ ảnh xám $128 \times 128 \Rightarrow$ vector đặc trưng kích thước 8100.
- Chuẩn hoá đặc trưng bằng `StandardScaler`.
- Giảm chiều dữ liệu bằng PCA với ngưỡng **95% phương sai** $\Rightarrow$ còn 128 chiều.
- Huấn luyện mô hình Random Forest trên đặc trưng sau bước PCA, sau đó dự đoán trên tập kiểm thử.

4.3.3 Huấn luyện mô hình MultiClassSVM One-vs-One

Stratified K-Fold Cross-Validation

Thay vì chia ngẫu nhiên dữ liệu thành k fold mà có thể làm mất cân bằng tỷ lệ nhãn trong từng fold, *StratifiedKFold* đảm bảo rằng tỉ lệ các lớp trong mỗi fold xấp xỉ tỉ lệ của toàn bộ tập dữ liệu. Điều này đặc biệt quan trọng khi dữ liệu bị lệch lớp (class imbalance), vì đánh giá trên fold thiếu đại diện có thể dẫn đến ước lượng hiệu năng cầu sai lệch.

- Với n_splits fold, phương pháp này chia dữ liệu thành n_splits nhóm sao cho mỗi nhóm giữ tỉ lệ nhãn tương tự tổng thể.
- Nếu `shuffle=True` và có `random_state`, việc phân chia được thực hiện ngẫu nhiên theo hạt giống cố định, cho phép tái lập (reproducibility).

- Giúp giảm phương sai của ước lượng cross-validation so với việc không stratify, đặc biệt khi số mẫu của một số lớp nhỏ.

Grid Search with Cross-Validation

GridSearchCV thực hiện tìm kiếm có hệ thống trên một lưới các tổ hợp siêu tham số (parameter grid). Mỗi tổ hợp tham số được đánh giá bằng cross-validation (ở đây dùng StratifiedKFold). Sau khi đánh giá toàn bộ tổ hợp, Phương pháp này chọn tổ hợp có điểm trung bình cross-validation cao nhất theo chỉ số đánh giá (scoring) đã chỉ định.

1. **Xác định lưới tham số:** ví dụ

$$\mathcal{G} = \{(C, \gamma) \mid C \in \{0.1, 1, 10, 100\}, \gamma \in \{\text{'scale'}, 0.01, 0.001\}\}.$$

2. **Chia dữ liệu:** với mỗi fold do StratifiedKFold tạo, chia tập huấn luyện thành một tập con huấn luyện và một tập con kiểm tra nội bộ (validation).
3. **Huấn luyện & đánh giá:** với mỗi cặp tham số trong $\mathcal{G}$:
 - Huấn luyện mô hình trên tập huấn luyện của fold.
 - Đánh giá mô hình trên tập validation theo hàm điểm đã chọn (ở đây `scoring='f1_macro'`).
4. **Tổng hợp kết quả:** tính điểm trung bình (và thường là độ lệch chuẩn) của chỉ số đánh giá qua tất cả các fold cho mỗi tổ hợp tham số.
5. **Chọn tham số tốt nhất:** tổ hợp có điểm trung bình cao nhất được chọn (và nếu `re-fit=True` thì mô hình được huấn luyện lại trên toàn bộ dữ liệu huấn luyện với tổ hợp tham số đó).

4.3.4 Huấn luyện Convolutinal Neural Networks

Các thông số huấn luyện

- Mô hình được huấn luyện trong **30 epoch** với **batch size = 32**.
- Sử dụng **learning rate = 0.001** cùng bộ tối ưu **Adam** để cập nhật trọng số.
- Hàm mất mát được chọn là **CrossEntropyLoss**, phù hợp cho bài toán phân loại nhiều lớp.
- Kích thước ảnh đầu vào được cố định ở **224×224×3**, tương thích với cấu trúc CNN của mô hình.

4.4 Evaluation Metrics

Khi xây dựng một mô hình Machine Learning, chúng ta cần một công cụ đánh giá xem mô hình hiệu quả như thế nào và làm cơ sở so sánh hiệu năng giữa các mô hình khác nhau. Hiệu năng của mô hình được đánh giá trên tập dữ liệu kiểm thử (*test set*). Gọi y_{pred} là vector dự đoán đầu ra khi đưa tập kiểm thử vào mô hình, y_{true} là vector nhãn thực tế tương ứng. Các thước đo đánh giá được tính toán dựa trên việc so sánh y_{pred} với y_{true} . Trong bài toán phân loại, một vài chỉ số đánh giá thường được sử dụng là **Accuracy**, **Precision**, **Recall**, **F1-Score** [5].

Khi thực hiện bài toán phân loại, có bốn trường hợp dự đoán có thể xảy ra:

- **True Positive (TP):** đối tượng thuộc lớp *Positive*, mô hình dự đoán vào lớp *Positive* (dự đoán đúng).
- **True Negative (TN):** đối tượng thuộc lớp *Negative*, mô hình dự đoán vào lớp *Negative* (dự đoán đúng).

- **False Positive (FP)**: đối tượng thuộc lớp *Negative*, mô hình dự đoán vào lớp *Positive* (dự đoán sai).
- **False Negative (FN)**: đối tượng thuộc lớp *Positive*, mô hình dự đoán vào lớp *Negative* (dự đoán sai).

4.4.1 Accuracy

Accuracy được tính bằng tỉ lệ giữa số mẫu được dự đoán đúng của tất cả các lớp và tổng số mẫu trong tập dữ liệu kiểm thử.

$$\text{Accuracy} = \frac{\sum_{i=1}^C \text{Số dự đoán đúng cho lớp } i}{\text{Tổng số mẫu}} \quad (1)$$

Trong đó: C là tổng số lớp.

Accuracy có ưu điểm là đơn giản, trực quan và thuận tiện cho việc so sánh giữa các mô hình. Đây là một chỉ số cho biết mức độ đúng đắn tổng thể của mô hình, đặc biệt hữu ích khi phân bố lớp cân bằng. Tuy nhiên, Accuracy rất nhạy với mất cân bằng lớp; không phân biệt được bản chất lỗi (FP hay FN) và không cho biết cụ thể mô hình hoạt động tốt hay kém ở lớp nào.

4.4.2 Precision

Precision được định nghĩa là tỉ lệ số mẫu true positive trong số những điểm được phân loại là positive (TP + FP).

Với từng lớp i , Precision được tính bởi:

$$\text{Precision}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FP}_i} \quad (2)$$

Chỉ số này trả lời cho câu hỏi: “Trong tổng số các mẫu mà mô hình đã gán nhãn là thuộc lớp i , có bao nhiêu phần trăm mẫu thực sự thuộc về lớp đó?” Một mô hình có Precision cao đảm bảo rằng khi nó dự đoán một mẫu thuộc lớp i , dự đoán đó có độ tin cậy cao.

4.4.3 Recall

Recall được định nghĩa là tỉ lệ số mẫu true positive trong số những điểm thực sự là positive (TP + FN).

$$\text{Recall}_i = \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i} \quad (3)$$

Chỉ số này trả lời cho câu hỏi: “Trong tổng số các mẫu thực sự thuộc lớp i , có bao nhiêu phần trăm được mô hình dự đoán đúng là lớp i ?” Một mô hình có Recall cao cho thấy khả năng bao phủ tốt các mẫu của lớp i .

4.4.4 F1-Score

Precision và Recall rất hữu ích trong trường hợp các lớp không được phân bố đồng đều. Để có thể kết hợp hài hòa giữa Precision và Recall, ta có thể tính điểm F-Score:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}} \quad (4)$$

Trong đó:

- $0 < \beta < 1$: ưu tiên *Precision* hơn *Recall*.

- $\beta > 1$: ưu tiên *Recall* hơn *Precision*.
- $\beta = 1$: cân bằng giữa *Precision* và *Recall*.

Khi $\beta = 1$, với mỗi lớp i , chỉ số trở thành F1-Score:

$$\text{F1-Score}_i = 2 \cdot \frac{\text{Precision}_i \cdot \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \quad (5)$$

F1-Score phản ánh mức cân bằng giữa *Precision* và *Recall*: chỉ số chỉ cao khi cả hai đều cao và giảm mạnh nếu một trong hai thấp.

4.5 Kết quả & đánh giá từng mô hình

4.5.1 Mô hình Naive Bayes

	precision	recall	f1-score	support
glioma	0.80	0.71	0.75	254
meningioma	0.65	0.51	0.57	306
no_tumor	0.72	0.92	0.81	140
pituitary	0.78	0.92	0.84	300
accuracy			0.74	1000
macro avg	0.74	0.76	0.74	1000
weighted avg	0.74	0.74	0.73	1000
Test Loss: 0.6883				
Overall Accuracy: 74.10%				

Bảng 1: Kết quả đánh giá mô hình Naive Bayes

- Độ chính xác tổng thể đạt khoảng **74%**, được xem là khá cao so với độ phức tạp thấp của mô hình *Naive Bayes*.
- Hai lớp `no_tumor` và `pituitary` được nhận dạng tốt, cho thấy mô hình có khả năng phân biệt các đặc trưng nổi bật giữa chúng.
- **Test loss** đạt giá trị thấp (**0.6883**), cho thấy xác suất dự đoán của mô hình tương đối ổn định và không quá sai lệch.
- Lớp `meningioma` có *recall* thấp (0.51), cho thấy mô hình bỏ sót khá nhiều mẫu thật. Nguyên nhân có thể do đặc trưng HOG chưa đủ mạnh để phân biệt rõ `meningioma` với `glioma` hoặc `pituitary`.
- Do *GaussianNB* giả định rằng các đặc trưng độc lập và tuân theo phân phối chuẩn, mô hình không thể nắm bắt các mối tương quan phức tạp giữa các pixel như các mô hình học sâu (*CNN*).
- Mặc dù đạt độ chính xác 74%, kết quả này vẫn còn thấp so với các mô hình học sâu (thường trên 90%) trong bài toán phân loại khối u não.

Ưu điểm:

- **Giả định độc lập:** Hoạt động tốt cho nhiều bài toán, miền dữ liệu và ứng dụng khác nhau.
- Mô hình **đơn giản nhưng hiệu quả**, đủ tốt để giải quyết nhiều bài toán như phân lớp văn bản, lọc thư rác (*spam filtering*), *sentiment analysis*, v.v.

- Cho phép **kết hợp tri thức tiên nghiệm** (*prior knowledge*) và **dữ liệu quan sát được** (*observed data*).
- Hoạt động tốt trong trường hợp có **sự chênh lệch về số lượng giữa các lớp phân loại**.
- Quá trình **huấn luyện mô hình nhanh**, việc ước lượng tham số đơn giản và không yêu cầu tài nguyên tính toán lớn.

Nhược điểm:

- **Giả định độc lập** giữa các đặc trưng – đây vừa là ưu điểm vừa là hạn chế, vì trong thực tế hầu hết các thuộc tính thường **phụ thuộc lẫn nhau**.
- Mô hình không được huấn luyện bằng các **phương pháp tối ưu mạnh và chặt chẽ** như trong các mô hình học sâu.
- Các tham số của mô hình chỉ là **ước lượng xác suất điều kiện đơn lẻ**, không tính đến sự tương tác giữa các đặc trưng.
- Do giả định đơn giản, hiệu suất của mô hình có thể giảm khi áp dụng cho các bài toán có **quan hệ phi tuyến hoặc phức tạp** giữa các đặc trưng.

Dự định cải tiến

- Thử nghiệm các biến thể khác của Naive Bayes
- Tối ưu hóa tham số trong PCA
- Tối ưu hóa tham số HOG
- Thêm bước chuẩn hóa hoặc rời rạc hóa đặc trưng
- Bổ sung tập xác thực (validation set)

4.5.2 Mô hình Random Forest

	precision	recall	f1-score	support
glioma	0.93	0.83	0.88	254
meningioma	0.86	0.88	0.87	306
no_tumor	0.97	0.99	0.98	140
pituitary	0.93	0.98	0.95	300
accuracy			0.91	1000
macro avg	0.92	0.92	0.92	1000
weighted avg	0.91	0.91	0.91	1000

Test Loss: 0.4130
Overall Accuracy: 91.20%

Bảng 2: Kết quả đánh giá mô hình Random Forest

- Mô hình đạt độ chính xác tổng thể 91%, cho thấy khả năng phân loại ổn định và hiệu quả cao đối với dữ liệu ảnh MRI não.
- Hai lớp **pituitary** và **no_tumor** đạt kết quả tốt nhất với F1-score lần lượt là 0.95 và 0.98, nhờ đặc trưng hình thái rõ ràng và ít bị nhầm lẫn với các lớp khác.
- Lớp **glioma** có recall thấp nhất (0.83), cho thấy mô hình vẫn bỏ sót một số mẫu thật (false negative), thường nhầm với lớp **meningioma**.

- Các chỉ số trung bình (macro và weighted) đều đạt khoảng 0.91-0.92, chứng tỏ mô hình hoạt động ổn định, không bị lệch về một lớp cụ thể.
- **Nhìn chung**, mô hình **Random Forest** cho thấy hiệu quả tốt và ổn định trong phân loại bốn loại khối u não.

Dự định cải tiến

- Tối ưu thêm tham số của Random Forest: Thử nghiệm với số lượng cây lớn hơn (`n_estimators` = 800–1000) và thay đổi độ sâu tối đa để cân bằng giữa độ chính xác và thời gian huấn luyện.
- Tối ưu hóa tham số HOG và PCA
- Bổ sung tập xác thực (validation set): Dùng để điều chỉnh tham số và theo dõi quá trình huấn luyện, tránh overfitting trong các lần thử nghiệm mới.
- Điều chỉnh trọng số lớp (`class_weight`): Áp dụng `class_weight` = "balanced" để cải thiện khả năng nhận dạng lớp khó *glioma*.

4.5.3 Mô hình MultiClassSVM One-vs-One

	precision	recall	f1-score	support
glioma	0.98	0.94	0.96	254
meningioma	0.95	0.96	0.95	306
no_tumor	0.97	0.99	0.98	140
pituitary	0.99	1.00	0.99	300
accuracy			0.97	1000
macro avg	0.97	0.97	0.97	1000
weighted avg	0.97	0.97	0.97	1000
Overall Accuracy: 97.00%				

Bảng 3: Kết quả đánh giá mô hình MultiClassSVM One-vs-One

- Độ chính xác (Accuracy) cao: Độ chính xác tổng thể của mô hình là 97% trên tổng số 1000 mẫu thử. Điều này có nghĩa là mô hình đã dự đoán đúng 970 trên 1000 mẫu, một tỷ lệ rất ấn tượng.
- Hiệu suất cân bằng: Cả hai chỉ số *macro avg* (trung bình cộng các chỉ số của từng lớp) và *weighted avg* (trung bình có trọng số, dựa trên số lượng mẫu của mỗi lớp) đều đạt 0.97 cho cả *precision*, *recall*, và *f1-score*.
- Không bị ảnh hưởng bởi mất cân bằng dữ liệu: Việc *macro avg* và *weighted avg* bằng nhau (0.97) cho thấy mô hình hoạt động tốt trên tất cả các lớp, bất kể sự chênh lệch về số lượng mẫu (*support*) giữa các lớp (ví dụ: lớp *no_tumor* chỉ có 140 mẫu trong khi *meningioma* có 306 mẫu).

4.5.4 Convolutional Neural Networks

- Mô hình đạt **độ chính xác tổng thể 90.6%** cho thấy khả năng phân loại ổn định.
- Hai lớp **pituitary** và **no_tumor** đạt kết quả cao nhất với F1-score lần lượt là **0.96** và **0.93**.
- Lớp **glioma** có *recall* thấp (0.80), mô hình còn bỏ sót một số mẫu thật, dễ nhầm với các loại u khác.
- Lớp **meningioma** có hiệu suất cân bằng giữa *precision* và *recall* (F1-score 0.87).

	precision	recall	f1-score	support
glioma	0.94	0.80	0.86	254
meningioma	0.84	0.90	0.87	306
no_tumor	0.89	0.97	0.93	140
pituitary	0.95	0.97	0.96	300
accuracy			0.91	1000
macro avg	0.91	0.91	0.91	1000
weighted avg	0.91	0.91	0.91	1000
Test Loss: 67.5466				
Overall Accuracy: 90.60%				

Bảng 4: Kết quả đánh giá mô hình CNN

- Các chỉ số trung bình (macro và weighted) đều khoảng **0.91**, chứng tỏ mô hình hoạt động ổn định và không lệch lớp.
- Nhìn chung, mô hình CNN đạt hiệu quả tốt cho bài toán phân loại khối u não, tuy nhiên cần cải thiện thêm ở lớp *glioma* bằng cách tăng dữ liệu hoặc tinh chỉnh tham số huấn luyện.

4.6 Tổng hợp kết quả thực nghiệm

Bảng dưới đây tóm tắt hiệu năng của bốn mô hình đã triển khai trên tập kiểm thử (Test set) của bộ dữ liệu BRISC.

Mô hình	Accuracy	Precision	Recall	F1-Score
Naive Bayes	74.10%	0.74	0.76	0.74
Random Forest	91.20%	0.91	0.91	0.91
CNN	90.60%	0.91	0.91	0.91
MultiClass SVM	97.00%	0.97	0.97	0.97

Bảng 5: Bảng tổng hợp hiệu năng các mô hình

Tổng hợp kết quả thực nghiệm cho thấy sự phân tầng rõ rệt về hiệu năng giữa các phương pháp tiếp cận, trong đó mô hình MultiClass SVM sử dụng chiến lược One-vs-One đạt kết quả vượt trội nhất với độ chính xác 97.0%, cho thấy sự hiệu quả của việc kết hợp các kỹ thuật HOG và PCA. Chiến lược One-vs-One đã giúp SVM chia nhỏ bài toán phân loại phức tạp thành nhiều bài toán nhị phân, từ đó tối ưu hóa được đường biên quyết định cho từng cặp lớp và duy trì sự cân bằng giữa các chỉ số Precision và Recall, khắc phục vấn đề mất cân bằng dữ liệu trong tập BRISC.

Một điểm đáng chú ý trong kết quả là sự tương đồng về hiệu năng giữa Random Forest (91.2%) và mô hình học sâu CNN tự thiết kế (90.6%). Mặc dù CNN thường được đánh giá cao hơn trong các tác vụ thị giác máy tính nhờ khả năng tự học đặc trưng, nhưng trong thực nghiệm này, nó không tạo ra khoảng cách lớn so với phương pháp học máy truyền thống, có thể là do kiến trúc thiết kế của nhóm chưa được tối ưu. Cả Random Forest và CNN đều gặp khó khăn chung trong việc nhận diện lớp *glioma*, với Recall thấp hơn hẳn so với các lớp còn lại, cho thấy đây là loại khối u có đặc điểm hình thái phức tạp và dễ gây nhầm lẫn nhất đối với cả hai phương pháp.

Ở vị trí cuối cùng, mô hình Naive Bayes chỉ đạt độ chính xác 74.1%, phản ánh rõ ràng hạn chế của giả định độc lập giữa các đặc trưng khi áp dụng vào dữ liệu hình ảnh. Trong ảnh MRI, mối tương quan không gian giữa các điểm ảnh lân cận mang thông tin cấu trúc quan trọng, điều mà



Naive Bayes đã bỏ qua dẫn đến việc không thể phân tách tốt các lớp có hình dạng khối u phức tạp như meningioma.

Tóm lại, thực nghiệm này khẳng định rằng đối với quy mô dữ liệu và bài toán cụ thể này, phương pháp MultiClass SVM là lựa chọn tốt nhất, mang lại độ chính xác cao và ổn định hơn so với các kiến trúc học sâu cơ bản.

5 Kết luận - Phân tích kết quả các mô hình đã sử dụng

5.1 Tổng kết kết quả đạt được

Đồ án đã hoàn thành mục tiêu xây dựng và triển khai hệ thống phân loại khối u não từ ảnh MRI sử dụng bộ dữ liệu **BRISC 2025**. Nhóm đã thực hiện nghiên cứu đa phương pháp, bao gồm cả các thuật toán học máy truyền thống và học sâu:

1. **Học máy truyền thống:** Triển khai thành công Naive Bayes, Random Forest và Multi-ClassSVM One-vs-One kết hợp với trích xuất đặc trưng HOG và giảm chiều dữ liệu PCA.
2. **Học sâu (Deep Learning):** Thiết kế và huấn luyện kiến trúc CNN tự đề xuất lấy cảm hứng từ ResNet với kỹ thuật skip connection.

Kết quả thực nghiệm trên 1000 mẫu thử cho thấy sự khác biệt rõ rệt về hiệu năng giữa các mô hình:

- **MultiClassSVM OVO:** Đạt độ chính xác cao nhất với **97.00%**.
- **Random Forest:** Đạt độ chính xác **91.20%**.
- **CNN (Kiến trúc tự thiết kế):** Đạt độ chính xác **90.60%**.
- **Naive Bayes:** Đạt độ chính xác thấp nhất với **74.10%**.

Bảng 6: So sánh hiệu năng giữa các mô hình phân loại khối u não trên bộ dữ liệu BRISC 2025

Mô hình	Đặc trưng / Kiến trúc	Độ chính xác (Accuracy)
MultiClassSVM OVO	HOG + PCA	97.00%
Random Forest	HOG + PCA	91.20%
CNN	ResNet-inspired (Skip connections)	90.60%
Naive Bayes	GaussianNB + PCA	74.10%

5.2 Nhận xét và Đánh giá

Thông qua quá trình thực nghiệm, nhóm rút ra các nhận định quan trọng sau:

1. **Hiệu quả vượt trội của SVM:** Trong bài toán này, MultiClassSVM OVO kết hợp đặc trưng HOG cho kết quả tốt nhất (97%). Điều này cho thấy với kích thước tập dữ liệu hiện tại, việc trích xuất đặc trưng thủ công dựa trên cấu trúc biên và hướng cạnh (HOG) vẫn mang lại giá trị rất lớn.
2. **Sự ổn định của Random Forest:** Mô hình đạt kết quả rất tốt trên các lớp có đặc điểm hình thái rõ ràng như *pituitary* và *no_tumor* (F1-score lên đến 0.98).
3. **Tiềm năng của CNN:** Mặc dù CNN tự thiết kế đạt 90.60%, thấp hơn SVM, nhưng mô hình này thể hiện ưu điểm ở khả năng tự động hóa mà không cần trích xuất đặc trưng thủ công. Hiệu suất lớp *glioma* còn thấp (recall 0.80) gợi ý rằng mô hình cần được tinh chỉnh thêm về kiến trúc hoặc áp dụng các kỹ thuật augmentation mạnh hơn để nhận diện các bất thường cấu trúc phức tạp.

4. **Hạn chế của Naive Bayes:** Giả định độc lập giữa các đặc trưng của Naive Bayes khiến mô hình khó nắm bắt được mối quan hệ không gian phức tạp giữa các pixel ảnh, dẫn đến recall thấp ở lớp *meningioma* (0.51).

5.3 Hướng phát triển tương lai

Để cải thiện hệ thống chẩn đoán hỗ trợ bác sĩ, các hướng phát triển tiếp theo bao gồm:

- Tối ưu hóa CNN: Thử nghiệm các kiến trúc học sâu tiên tiến hơn như Vision Transformers (ViT) hoặc sử dụng Transfer Learning từ các mô hình đã được huấn luyện trên dữ liệu y tế quy mô lớn.
- Ensemble Learning: Kết hợp kết quả dự đoán từ SVM và CNN để tận dụng thế mạnh của cả đặc trưng thủ công và đặc trưng tự động.
- Giải thích mô hình (Explainable AI): Áp dụng kỹ thuật Grad-CAM để trực quan hóa các vùng mà mô hình tập trung vào khi đưa ra dự đoán, giúp tăng độ tin cậy trong môi trường lâm sàng.
- Mở rộng tập dữ liệu: Thu thập thêm ảnh MRI từ các nguồn khác nhau để kiểm tra tính tổng quát của mô hình trước các biến thể về nhiễu và độ sáng của thiết bị chụp.



Tài liệu

- [1] Leo Breiman. Random forests. *Machine Learning*, 2001.
- [2] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.
- [3] Navneet Dalal and Bill Triggs. Histograms of oriented gradients for human detection. In *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR'05)*, volume 1, pages 886–893. Ieee, 2005.
- [4] Bana Falakhi, Elmira Faustina Achmal, Muhammad Rizaldi, Renata Rizki Rafi'Athallah, and Novanto Yudistira. Perbandingan model alexnet dan resnet dalam klasifikasi citra bunga memanfaatkan transfer learning. *Jurnal Ilmu Komputer dan Agri-Informatika*, 9(1):70–78, 2022.
- [5] Amirreza Fateh, Yasin Rezvani, Sara Moayed, Sadjad Rezvani, Fatemeh Fateh, Mansoor Fateh, and Vahid Abolghasemi. Brisc: Annotated dataset for brain tumor segmentation and classification with swin-hafnet. *arXiv preprint arXiv:2506.14318*, 2025.
- [6] Michael Greenacre, Patrick JF Groenen, Trevor Hastie, Alfonso Iodice d'Enza, Angelos Markos, and Elena Tuzhilina. Principal component analysis. *Nature Reviews Methods Primers*, 2(1):100, 2022.
- [7] Trevor Hastie and Robert Tibshirani. Classification by pairwise coupling. *Advances in neural information processing systems*, 10, 1997.
- [8] Tin Kam Ho. The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(8):832–844, 1998.
- [9] Chih-Wei Hsu and Chih-Jen Lin. A comparison of methods for multiclass support vector machines. *IEEE transactions on Neural Networks*, 13(2):415–425, 2002.
- [10] Qingyong Hu, Bo Yang, Linhai Xie, Stefano Rosa, Yulan Guo, Zhihua Wang, Niki Trigoni, and Andrew Markham. Randla-net: Efficient semantic segmentation of large-scale point clouds. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11108–11117, 2020.
- [11] Brett Koonce. Vgg network. In *Convolutional Neural Networks with Swift for Tensorflow: Image Recognition and Dataset Categorization*, pages 35–50. Springer, 2021.
- [12] A Emin Orhan and Xaq Pitkow. Skip connections eliminate singularities. *arXiv preprint arXiv:1701.09175*, 2017.
- [13] Bernhard Schölkopf and Alexander J Smola. *Learning with kernels: support vector machines, regularization, optimization, and beyond*. MIT press, 2002.
- [14] Sasha Targ, Diogo Almeida, and Kevin Lyman. Resnet in resnet: Generalizing residual architectures. *arXiv preprint arXiv:1603.08029*, 2016.
- [15] Hữu Tiệp Vũ. *Machine Learning cơ bản*. 2018. Online book.
- [16] Jingsi Zhang, Xiaosheng Yu, Xiaoliang Lei, and Chengdong Wu. A novel deep lenet-5 convolutional neural network model for image recognition. *Computer Science and Information Systems*, 19(3):1463–1480, 2022.
- [17] Tong Zhang. An introduction to support vector machines and other kernel-based learning methods. *Ai Magazine*, 22(2):103–103, 2001.